

# FICHE MÉMO TECHNIQUE — BLOC 4 : AT&T SMS SPAM DETECTOR

Filtrage Intelligent Télécom & Architecture Deep Learning Bi-LSTM (Keras / TensorFlow)

Candidat : Christopher GILLERON  
Certification CSDS (RNCP 35288)

## 1. CADRAGE MÉTIER & MORPHOLOGIE DES DONNÉES

BUSINESS

**Enjeu AT&T** : Protection contre le *smishing* (hameçonnage SMS bancaire, faux colis, escroqueries). Contrainte industrielle à **coût d'erreur asymétrique** :

- **Faux Négatif (FN)** : Spam non détecté qui passe → simple désagrément pour l'abonné qui le supprime.
- **Faux Positif (FP)** : SMS légitime bloqué (code 2FA bancaire, validation de carte, urgence médicale) → **CRITIQUE & INACCEPTABLE** (rupture de service, churn client direct).

**Dataset AT&T (5 572 SMS)** :

- **Ham (Légitimes)** : 4 825 SMS (**86,6 %**). Courts (moy. 71 car., écart-type  $\sigma = 58$ ), structure conversationnelle libre.
- **Spam (Indésirables)** : 747 SMS (**13,4 %**). Calibrés et denses (moy. 139 car., écart-type  $\sigma = 29$ ), optimisés pour la limite SMS standard (160 car.), saturés d'impératifs (*call, win, free, claim*).
- **Split & Stratification** : Partition Train (80% = 4 457) / Test (20% = 1 115) avec `stratify=y` pour maintenir rigoureusement le ratio 86,6% / 13,4% sur les deux sous-ensembles.

**Question Jury** : Pourquoi l'Accuracy globale est-elle trompeuse ici ?

**Réponse** : Avec 86,6% de Hams, un modèle naïf prédisant systématiquement 0 (Ham) affiche 86,6% d'accuracy sans jamais intercepter le moindre spam.

## 2. PRÉTRAITEMENT NLP KERAS & TOKENISATION

PIPELINE

Contrairement au One-Hot ou au TF-IDF (sac de mots statique), le modèle séquentiel Keras exige une représentation ordonnée :

**A. Tokenizer Keras (fit\_on\_texts sur Train uniquement)**

- `num_words=10000` : Limite le dictionnaire aux 10 000 mots les plus fréquents (équilibre exhaustivité / mémoire).
- `oov_token="<OOV>"` (Out-Of-Vocabulary) : Attribue un index spécial (ex: 1) à tout mot inconnu rencontré lors des prédictions en production, évitant tout crash logiciel.

**B. Padding & Truncating (pad\_sequences)**

- `maxlen=100` : Longueur fixe de la séquence en entrée du réseau de neurones.
- `padding="post"` : Complète les SMS de moins de 100 tokens par des zéros à la fin.
- `truncating="post"` : Tronque les SMS dépassant 100 tokens en conservant le début.
- **Matrice d'entrée finale** : dimensions (`batch_size, 100`) contenant des entiers.

**Question Jury** : Pourquoi fit le tokenizer uniquement sur le train set ?

**Réponse** : Pour éviter la fuite d'information (**Data Leakage**). Le modèle ne doit avoir aucune connaissance préalable sur le vocabulaire ou la fréquence des mots du test set.

## 3. COUCHE D'EMBEDDING (ESPACE DENSE)

CONCEPTS

`Embedding(input_dim=10000, output_dim=32, mask_zero=True)`

- **Rôle** : Projette chaque identifiant entier d'un mot dans un espace vectoriel continu dense à **32 dimensions**.
- **Apprentissage supervisé** : Contrairement à Word2Vec ou GloVe pré-entraînés, les poids de l'embedding sont ajustés par rétropropagation directement sur la tâche de détection de spam.
- **Sémantique vectorielle** : Rapproche mathématiquement les synonymes malveillants (les vecteurs de `cash`, `prize` et `winner` ont une forte proximité cosinus).
- `mask_zero=True` : Indique aux couches récurrentes d'ignorer les zéros de padding, économisant du calcul et évitant de biaiser la mémoire du LSTM.

## 4. LE RÉSEAU RÉCURRENT BI-DIRECTIONNEL (BI-LSTM)

DEEP LEARNING

`Bidirectional(LSTM(units=32, return_sequences=False))`

**A. Pourquoi un LSTM plutôt qu'un RNN simple ?**

Les RNN classiques souffrent de la **disparition du gradient (Vanishing Gradient)** : la rétropropagation sur des séquences longues multiplie des gradients  $< 1$ , empêchant d'apprendre des dépendances éloignées. La cellule LSTM résout cela via une **cellule d'état mémoire interne ( $C_t$ )** régulée par 3 portes (gates) :

- **Forget Gate ( $f_t$ )** :  $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \rightarrow$  Décide quelle fraction de l'information passée doit être oubliée.
- **Input Gate ( $i_t$ )** : Filtre la nouvelle information candidate à écrire dans la mémoire.
- **Output Gate ( $o_t$ )** : Calcule le nouvel état caché  $h_t$  émis en sortie ( $h_t = o_t \cdot \tanh(C_t)$ ).

**B. Pourquoi la Bidirectionnalité (Bi-LSTM) ?**

Comporte **deux sous-réseaux LSTM indépendants** : l'un lit de gauche à droite (passé  $\rightarrow$  futur), l'autre de droite à gauche (futur  $\rightarrow$  passé). Leurs états cachés sont concaténés en sortie ( $32 + 32 = 64$  descripteurs).

*Exemple Smishing* : Dans "Urgent, votre compte est suspendu, cliquez sur ce lien", l'urgence au début ne prend son sens malveillant qu'à la lecture du lien final, et inversement.

## 5. COUCHES DENSES, DROPOUT & SIGMOÏDE

INFÉRENCE

Le bloc de décision final se compose de :

`Dense(32, activation="relu")  $\rightarrow$  Dropout(0.20)  $\rightarrow$  Dense(1, activation="sigmoid")`

**1. Dense(32, activation='relu') : Couche entièrement connectée**

- Reçoit le vecteur de 64 features issu du Bi-LSTM.
- Applique une transformation affine  $z = Wx + b$  suivie de l'activation **ReLU** :  $\text{ReLU}(z) = \max(0, z)$ .
- **Rôle** : Combine de manière non-linéaire les signaux de contexte extraits par le LSTM et accélère l'apprentissage en évitant la saturation pour  $z > 0$ .

**2. Dropout(0.20) : Régularisation par désactivation stochastique**

- Désactive aléatoirement **20% des neurones** à chaque passe forward d'entraînement (met leurs sorties à 0).
- **Rôle** : Empêche la co-adaptation complexe des neurones (force le réseau à trouver des représentations redondantes robustes). Élimine le surapprentissage (**Overfitting**). Désactivé à l'inférence.

**3. Dense(1, activation='sigmoid') : Neurone de sortie binaire**

- Un seul neurone scalaire avec activation **Sigmoïde** :  $\sigma(z) = 1 / (1 + e^{-z})$ .
- **Sortie** : Délivre une probabilité calibrée  $P(\text{Spam} | \text{SMS})$  comprise entre 0 et 1.
- Si  $P \geq \text{seuil}$  (ex: 0,50), la classe prédite est `Spam (1)`, sinon `Ham (0)`.

## 6. ENTRAÎNEMENT, OPTIMISEUR & LOSS

TRAINING

- **Loss (Perte)** : `binary_crossentropy` :  $\text{Loss} = - (1/N) \sum [y \cdot \ln(p) + (1 - y) \cdot \ln(1 - p)]$ . Mesure la divergence entre la probabilité prédite  $p$  et l'étiquette réelle  $y \in \{0, 1\}$ .
- **Optimiseur Adam (Adaptive Moment Estimation)** : Taux d'apprentissage  $10^{-3}$  (0,001). Combine l'inertie (momentum) et l'adaptation du pas par gradient quadratique (RMSProp).
- **Total Paramètres** : 338 753 paramètres entraînaibles (dont 320 000 pour l'Embedding  $10\,000 \times 32$ ). Modèle très léger ( $\sim 1,3$  Mo).
- **EarlyStopping** : `monitor="val_loss", patience=3, restore_best_weights=True`. Interrompt la descente de gradient dès que la perte de validation cesse de décroître pendant 3 epochs, restaurant automatiquement les meilleurs poids.

7. DÉFINITION & FORMULES FORMELLES DES MÉTRIQUES

MATHÉMATIQUES

Sur le jeu de test de 1 115 SMS (966 Hams, 149 Spams) :

A. Matrice de Confusion (Terminologie Télécom)

- Vrais Négatifs (TN)** : Hams correctement identifiés et délivrés à l'abonné.
- Faux Positifs (FP)** : Hams légitimes bloqués à tort (code bancaire perdu).
- Faux Négatifs (FN)** : Spams manqués qui atterrissent sur le smartphone.
- Vrais Positifs (TP)** : Spams correctement détectés et interceptés.

B. Accuracy (Exactitude Globale)

Accuracy =  $\frac{TP + TN}{TP + TN + FP + FN}$  = (Prédictions Correctes / Total)

Pourcentage total de bonnes classifications. *Faiblesse* : insensible à la gravité des erreurs sur classe minoritaire.

C. Précision (Spam) — Fiabilité de l'Alarme

Précision =  $\frac{TP}{TP + FP}$  = (Spams Confirmés / Alertes Totales)

"Quand le filtre dit Spam, quelle est la certitude que c'en est un ?" Plus elle est haute, moins on risque de bloquer un SMS authentique d'un client.

D. Rappel / Recall / Sensibilité (Spam) — Taux d'Interception

Rappel =  $\frac{TP}{TP + FN}$  = (Spams Détectés / Total Réel des Spams)

"Sur 100 spams en circulation, quelle proportion le filtre intercepte-t-il ?"

E. F1-Score (Spam) — Moyenne Harmonique

F1-Score =  $2 \times \frac{Précision \times Rappel}{Précision + Rappel}$  =  $\frac{2 \cdot TP}{2 \cdot TP + FP + FN}$

Sanctionne sévèrement si l'une des deux métriques s'effondre (alors qu'une moyenne arithmétique masquerait un recall catastrophique).

F. ROC-AUC (Area Under the ROC Curve)

Intègre le taux de vrais positifs (TPR) en fonction du taux de faux positifs (FPR) pour *tous les seuils possibles de 0 à 1*. Mesure la capacité intrinsèque du modèle à ordonner les probabilités (1.0 = séparateur parfait, 0.5 = hasard).

8. BENCHMARK COMPARATIF : NOTEBOOK & PPTX

RÉSULTATS

Chiffres réels vérifiés issus du jeu de test (1 115 SMS, seuil standard 0,50) :

| Métrique           | Baseline (TF-IDF + LogReg) | Champion (Bi-LSTM Keras) | Impact Métier                          |
|--------------------|----------------------------|--------------------------|----------------------------------------|
| Accuracy           | 97,13 %                    | 98,57 %                  | +1,44 pt global                        |
| Précision (Spam)   | 99,16 %                    | 95,24 %                  | Excellente fiabilité                   |
| Recall (Spam)      | 79,19 %                    | 93,96 %                  | +14,77 pts de spams captés             |
| F1-Score (Spam)    | 0,8806                     | 0,9459                   | +0,065 (+6,5 pts)                      |
| ROC-AUC            | 0,9868                     | 0,9925                   | Séparation quasi-parfaite              |
| Faux Négatifs (FN) | 31 spams ratés             | 9 spams ratés            | Volume de fuite divisé par 3,4         |
| Faux Positifs (FP) | 1 ham bloqué               | 7 hams bloqués           | Ajustable via le seuil (cf. ci-contre) |

Synthèse : La baseline linéaire laissait passer 1 spam sur 5 (31 ratés). Le Bi-LSTM divise ce volume d'échec par plus de 3 tout en maintenant le taux de faux positifs sous la barre de 0,7%.

9. CURSEUR DE SEUIL (THRESHOLD MOVING)

OPÉRATIONS

Le seuil par défaut de 0,50 n'est qu'une convention. En télécom, le seuil est un  **curseur d'arbitrage financier et de sécurité**  :

A. Arbitrage Précision vs Rappel

- Augmenter le seuil (0 → 0,70)** : On exige une très haute certitude pour déclarer un message comme Spam. Les Faux Positifs chutent à ≤ 2 (priorité 0 client légitime lésé), mais quelques spams passent.
- Diminuer le seuil (0 → 0,30)** : Le filtre devient agressif. Le rappel grimpe à >97%, mais le risque de faux positifs augmente.

B. Politiques Opérationnelles Identifiées

| Politique            | Seuil | F1-Score | FP (Hams bloqués) | FN (Spams ratés) |
|----------------------|-------|----------|-------------------|------------------|
| Standard (0,50)      | 0,50  | 0,946    | 7                 | 9                |
| Optimal F1 & Zéro FP | 0,67  | 0,959    | 2 (0,2 %)         | 10               |

C. Recommandation Architecture en Production

Pour concilier les deux contraintes, mise en place d'un système à **trois zones de routage** :

- P(Spam) < 0,30 → **Boîte de réception directe** (Délivrance immédiate).
- 0,30 ≤ P(Spam) ≤ 0,70 → **Dossier 'Courrier Indésirable / Quarantaine'** (l'abonné peut consulter sans blocage bloquant).
- P(Spam) > 0,70 → **Blocage automatique réseau** (Menace avérée).

10. QUESTIONS FRÉQUENTES DU JURY (Q&A SOUTENANCE)

JURY Q&A

**Q1 : Pourquoi ne pas avoir utilisé un modèle pré-entraîné comme BERT ou RoBERTa ?**

**R** : Les modèles de type BERT comportent plus de 110 millions de paramètres et nécessitent des GPU puissants générant 50 à 100 ms de latence par requête. Notre modèle Bi-LSTM ne pèse que **338k paramètres (~1,3 Mo)** et s'exécute en **< 5 ms par SMS sur simple CPU**, ce qui est compatible avec le débit de traitement en temps réel d'un opérateur mobile (ou embarquable via TensorFlow Lite).

**Q2 : Comment gérez-vous le "Leetspeak" (ex: "fr33", "w1n") et les fautes volontaires ?**

**R** : Le tokenizer convertit les termes inconnus en jeton `<OOV>`. De plus, le Bi-LSTM s'appuie sur le *contexte syntaxique environnant* (mots adjacents, structure de la phrase) plutôt que sur un mot isolé. Une perspective d'amélioration est l'encodage au niveau des sous-mots (Byte-Pair Encoding) ou au niveau caractères (Character-level CNN/LSTM).

**Q3 : Pourquoi avoir choisi Adam et non une descente de gradient classique (SGD) ?**

**R** : Adam calcule des taux d'apprentissage individuels adaptatifs pour chaque paramètre à partir des moments d'ordre 1 (moyenne) et d'ordre 2 (variance) des gradients. Il converge beaucoup plus vite que SGD sur des données textuelles éparées et évite de rester bloqué dans des plateaux.

**Q4 : Quels artefacts sont déployés dans votre application Streamlit (app.py) ?**

**R** : Deux fichiers sérialisés : `att_spam_model.keras` (architecture et poids entraînés du réseau) et `tokenizer.pickle` (dictionnaire d'apprentissage des mots et index). L'application charge ces fichiers au démarrage, applique le même padding, et permet de faire varier interactivement le seuil de décision pour visualiser l'impact métier en direct.